

Motion Planning Lecture 7

RRT, RRT*, kinodynamic RRT

Wolfgang Hönig (TU Berlin) and Andreas Orthey (Realtime Robotics)

May 28, 2025

Recap

Foundations

2 Weeks (problem formulation, terminology, collision checking)

Search-based

2 Weeks (A* and variants; state-lattice-based planning)

Sampling-based

5 Weeks (RRT, PRM, OMPL, Sampling Theory)

Optimization-based

2 Weeks (SCP, TrajOpt)

Current and Advanced Topics

3 Weeks (Comparative Analysis, Hybrid- and Multi-Robot approaches)

Today

- Geometric tree-based motion planning (RRT)
- Introduction asymptotically optimal planning
- Optimal geometric tree-based planning (RRT*)
- Kinodynamic tree-based motion planning (kinodynamic RRT)

Tree-based motion planning

Rapidly-exploring random tree (RRT)

- Invented independently by Steve M. LaValle (1998) and David Hsu (1997)
- One of the most efficient algorithms for motion planning
- Growing a tree through random extensions

D Hsu, et al., "Path planning in expansive configuration spaces" (1997)

SM LaValle, "Rapidly-exploring random trees: A new tool for path planning", (1998)

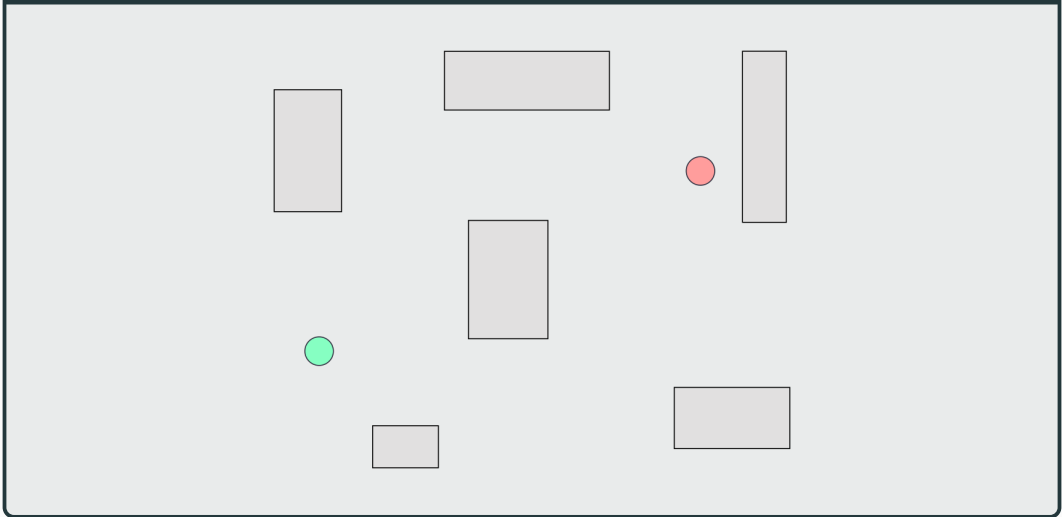
JJ Kuffner, SM LaValle, "RRT-connect: An efficient approach to single-query path planning", (2000)

Pseudocode RRT

```
1  def RRT(xstart, xgoal, mu):  
2      V.AddNode(xstart)  
3      while not finished:  
4          xrand = SampleRandom()  
5          xnear = NearestNeighbor(xrand)  
6          xnew = Steer(xnear, xrand, mu)  
7          if xnear == xnew:  
8              continue  
9          V.AddNode(xnew)  
10         V.AddEdge(xnear, xnew)  
11         if Distance(xnew, xgoal) < Epsilon:  
12             return Path(xnew)
```

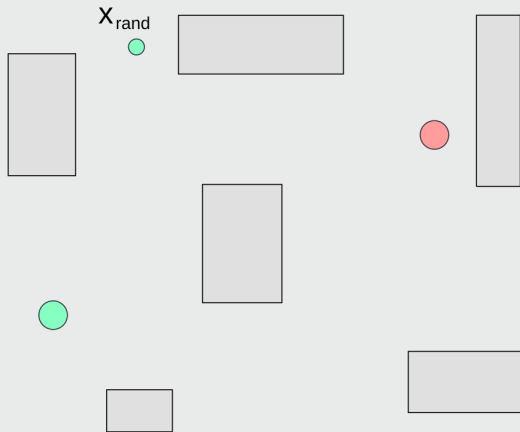
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



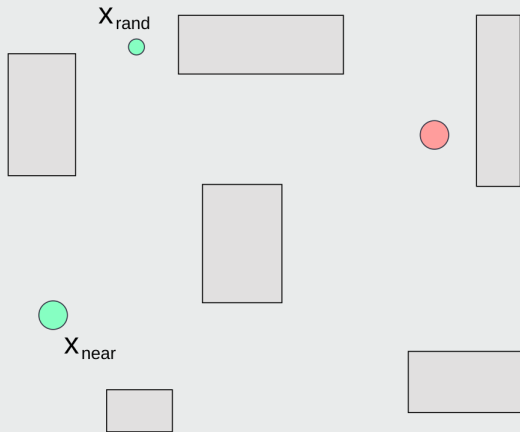
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



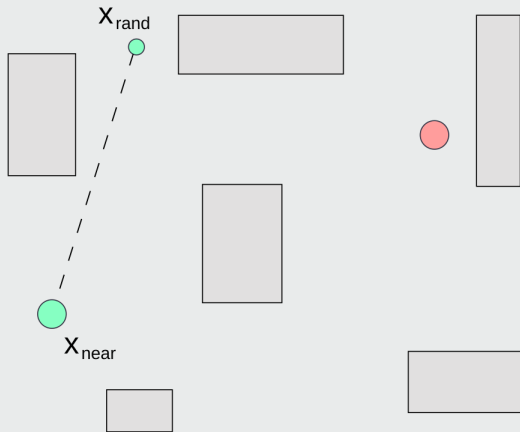
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



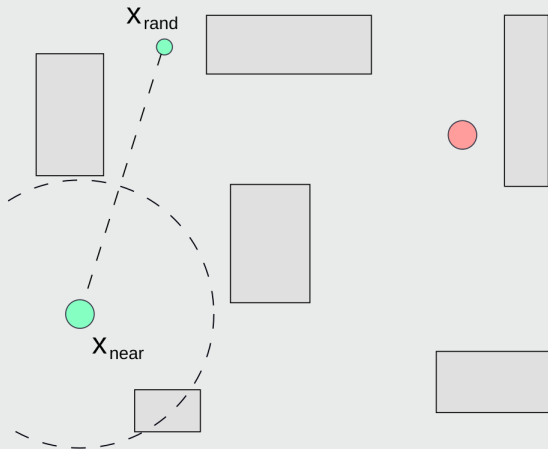
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



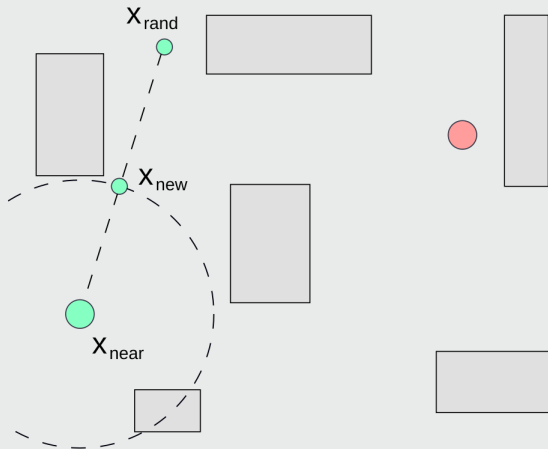
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



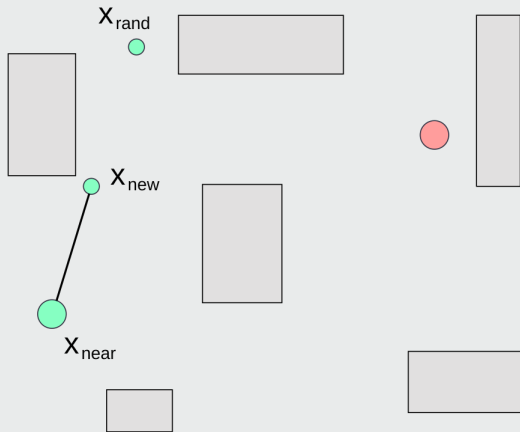
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



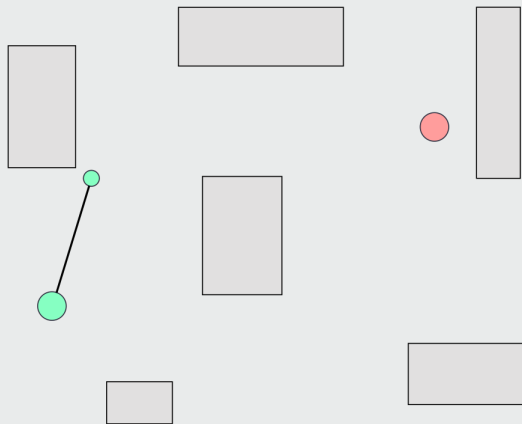
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



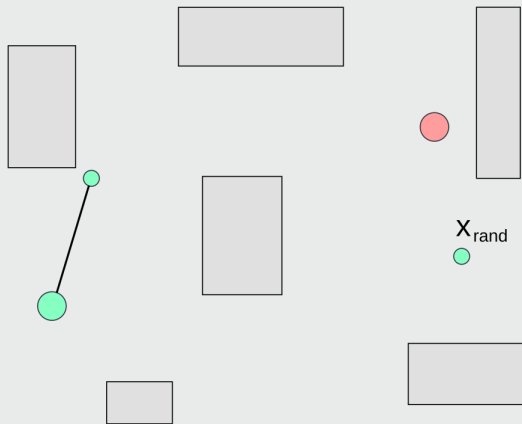
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

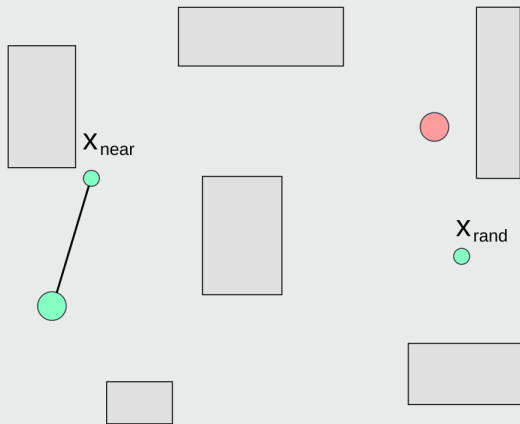


Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

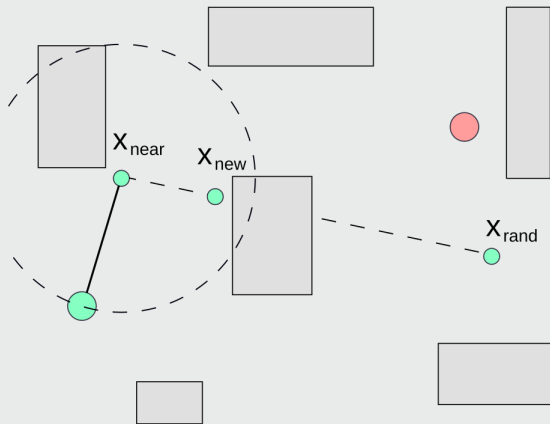


Rapidly-exploring random tree (RRT)



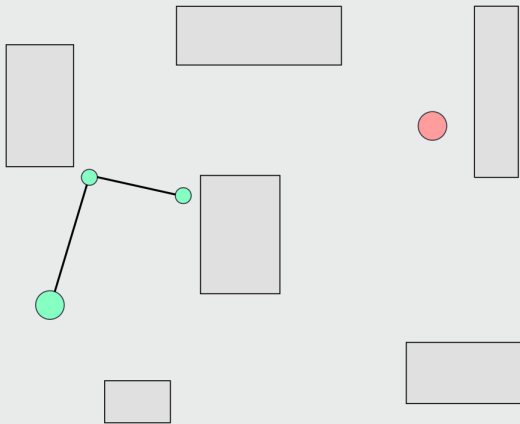
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



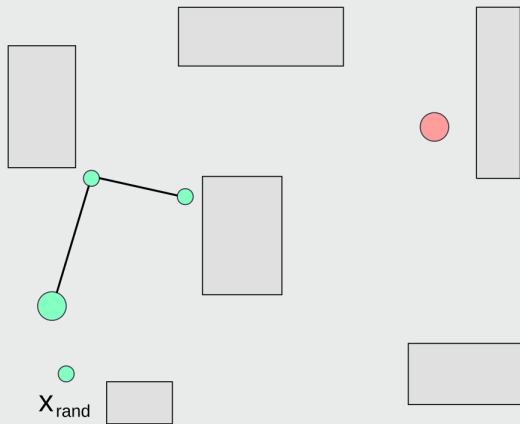
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



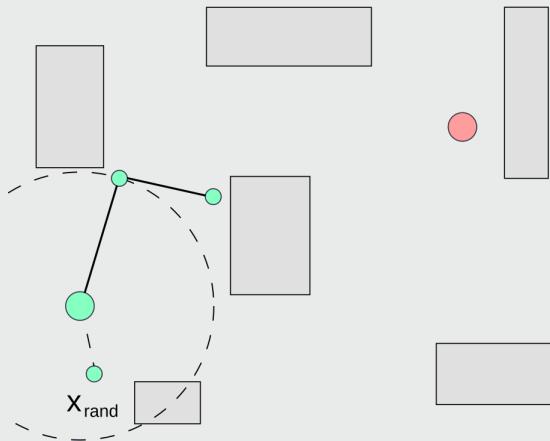
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

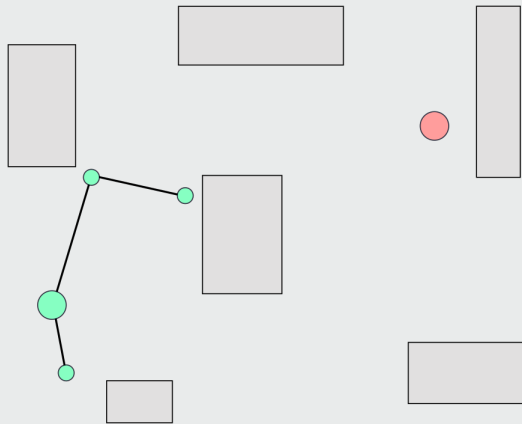


Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

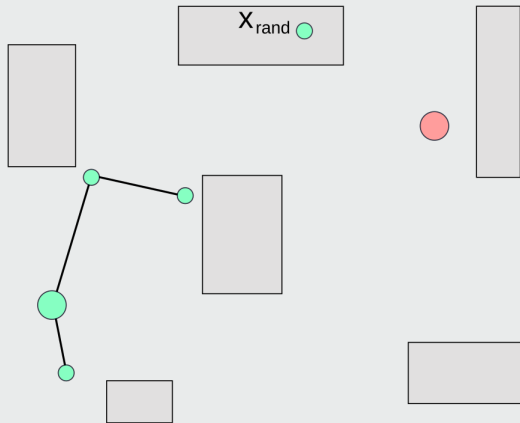


Rapidly-exploring random tree (RRT)



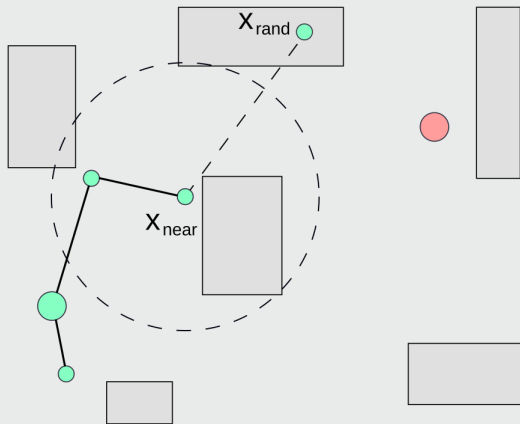
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



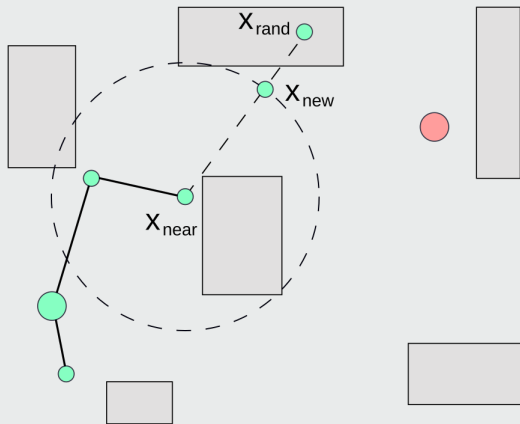
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

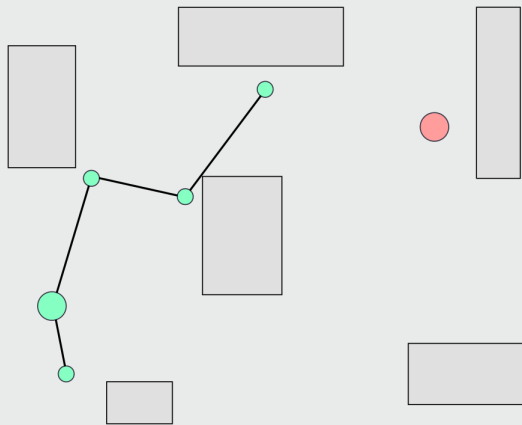


Rapidly-exploring random tree

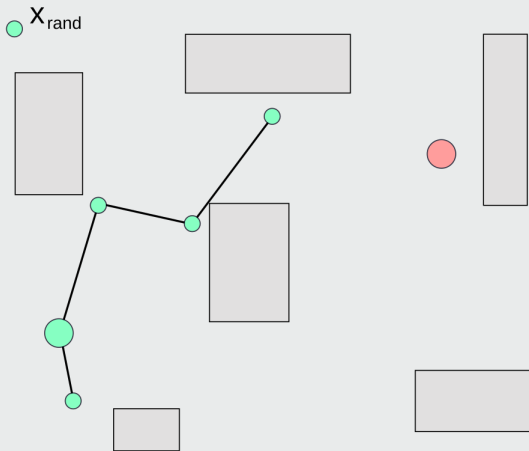
Rapidly-exploring random tree (RRT)



Rapidly-exploring random tree (RRT)

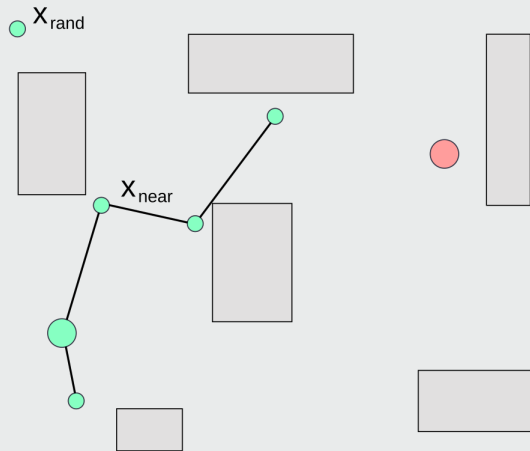


Rapidly-exploring random tree (RRT)



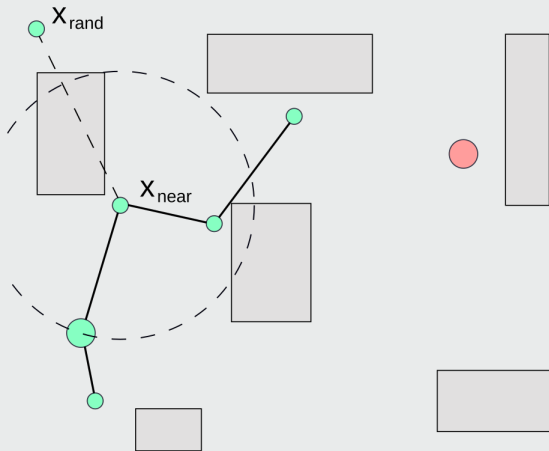
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



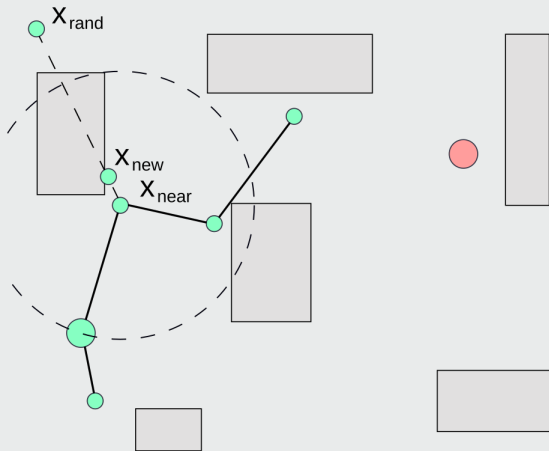
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

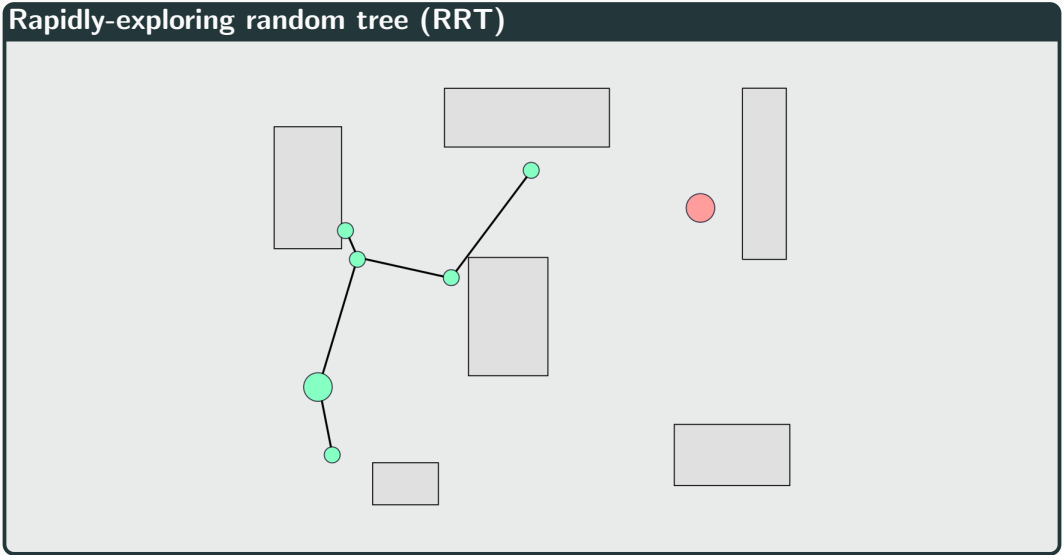


Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

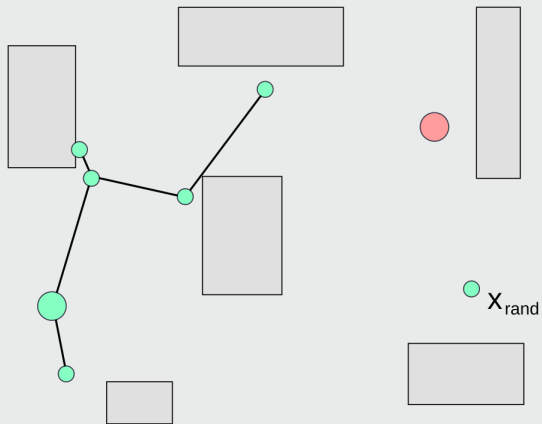


Rapidly-exploring random tree (RRT)



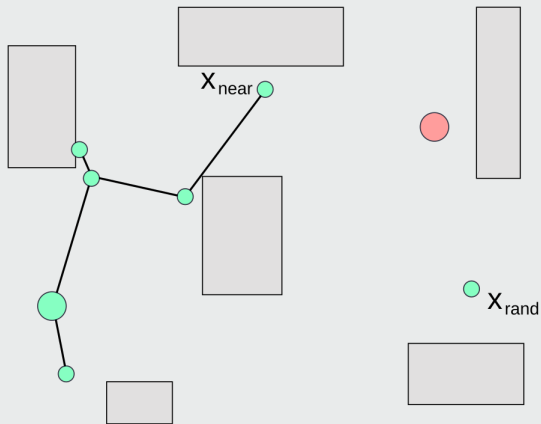
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



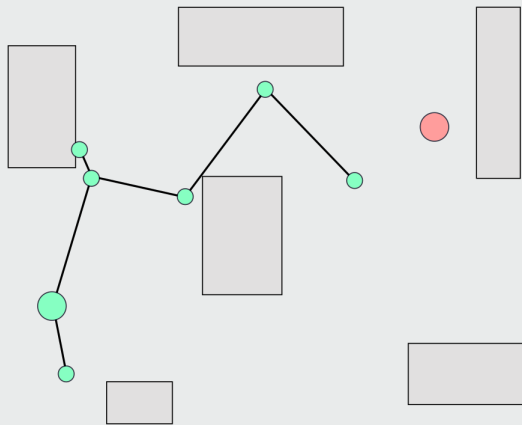
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



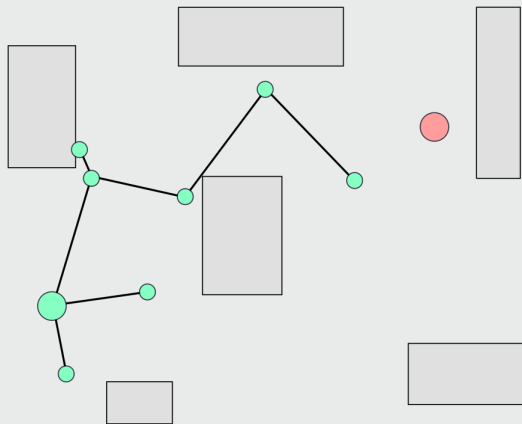
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)

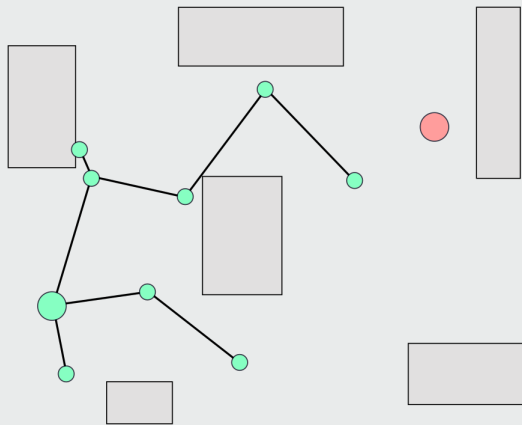


Rapidly-exploring random tree

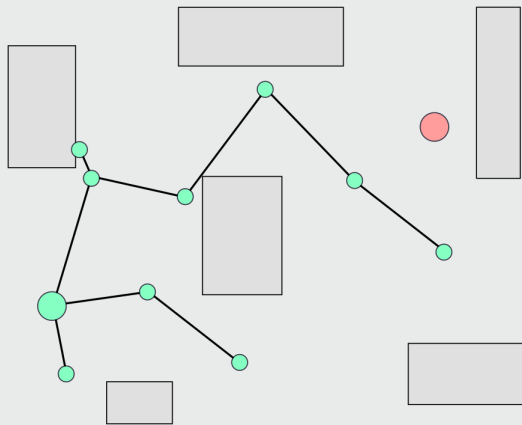
Rapidly-exploring random tree (RRT)



Rapidly-exploring random tree (RRT)

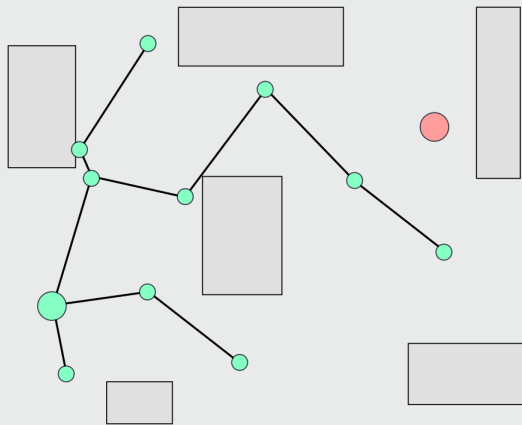


Rapidly-exploring random tree (RRT)



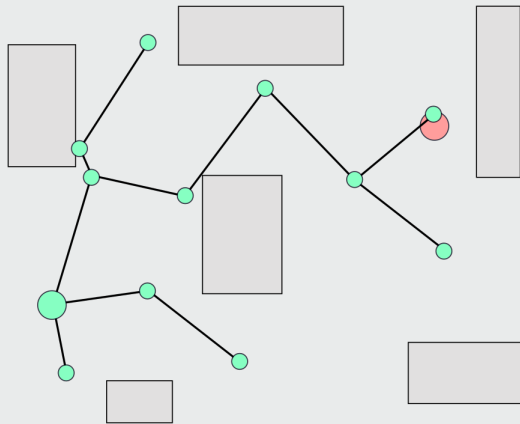
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



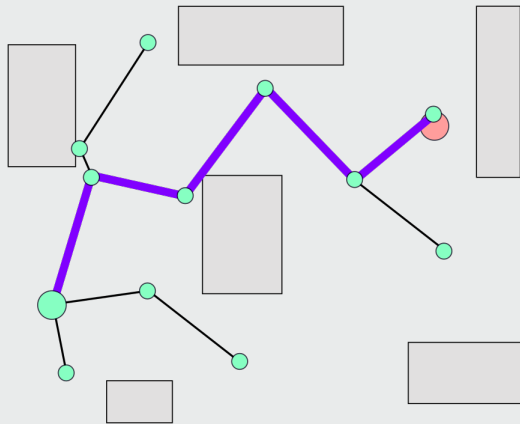
Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



Rapidly-exploring random tree

Rapidly-exploring random tree (RRT)



Rapidly-exploring random tree (RRT)

Efficiency

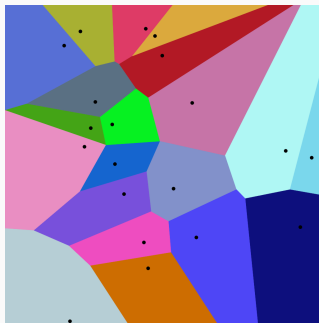
RRT is one of the most efficient planners because it has an implicit Voronoi bias

Rapidly-exploring random tree (RRT)

Voronoi region

Let $q_1, \dots, q_K$ be a set of configurations on the state space $\mathcal{Q}$. The Voronoi region is defined as

$$R_k = \{q \in \mathcal{Q} \mid d(q, R_k) \leq d(q, R_j), \text{ for all } j \neq k\}$$



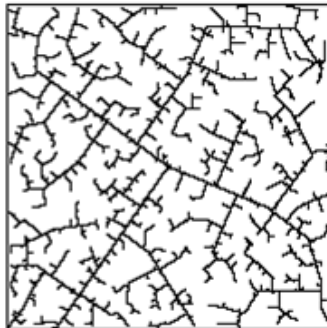
Rapidly-exploring random tree (RRT)

Voronoi bias

Probability of being selected is proportional to Voronoi region of a node in the tree.
Exploration/Exploitation trade-off.



45 iterations



390 iterations

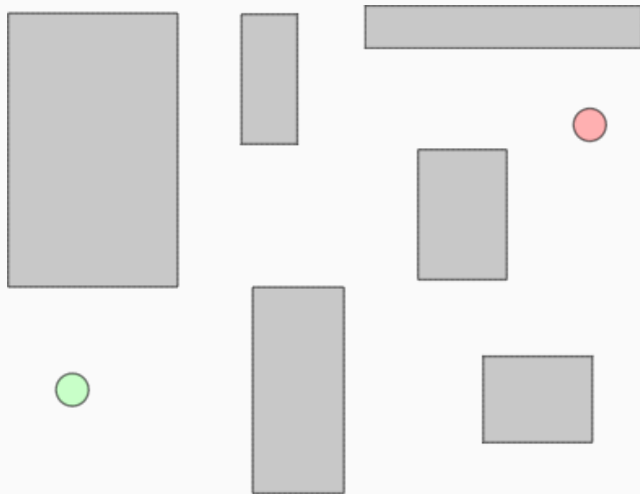
Tree-based motion planning

Probabilistic completeness proof RRT

Probabilistic Completeness RRT

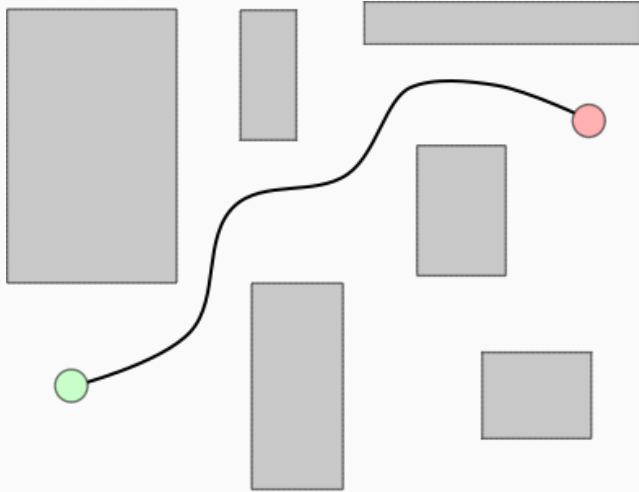
- A planner is probabilistic complete if it finds a solution if one exists.
- Main proof for RRT is based on induction.
- Requires number of samples going to infinity.

Proof sketch



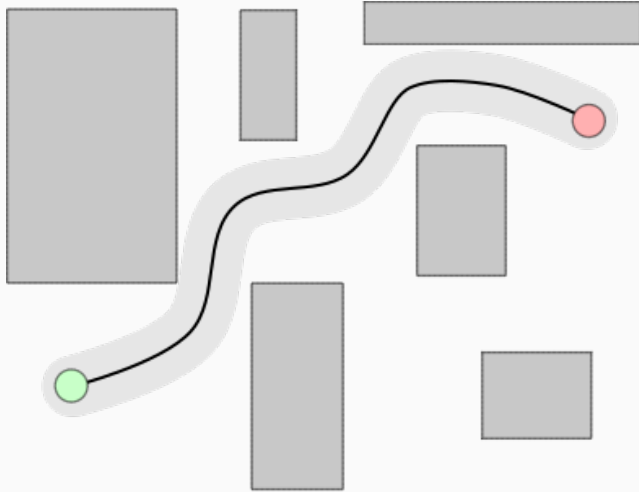
Petr Svestka, "On Probabilistic Completeness and Expected Complexity of Probabilistic Path Planning", 1998

Proof sketch



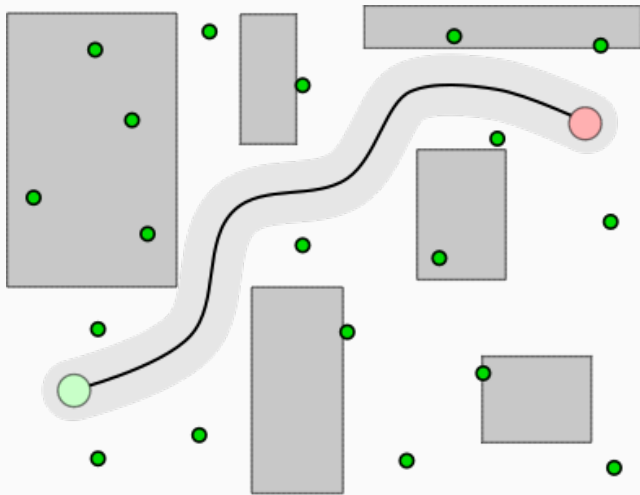
Assumption A: There exists a feasible path.

Proof sketch



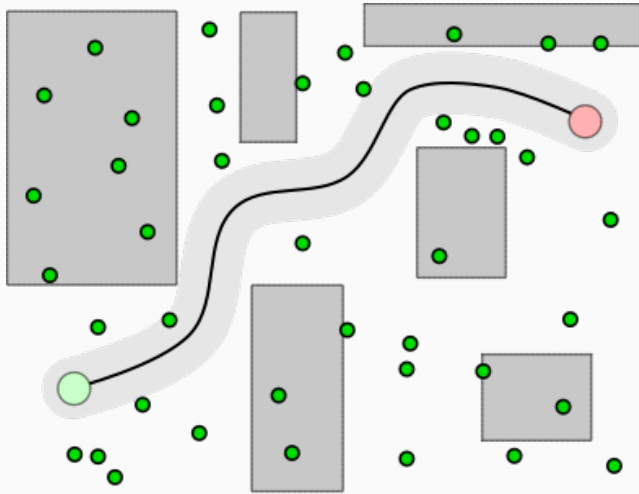
Assumption B: Feasible path has ϵ clearance.

Proof sketch



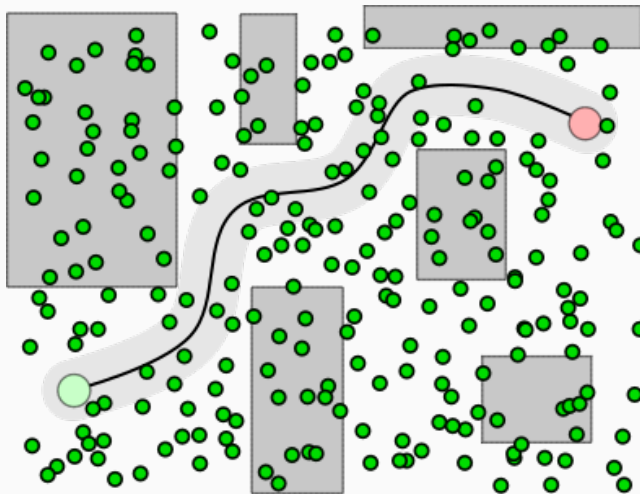
Assumption C: Sampling is dense.

Proof sketch



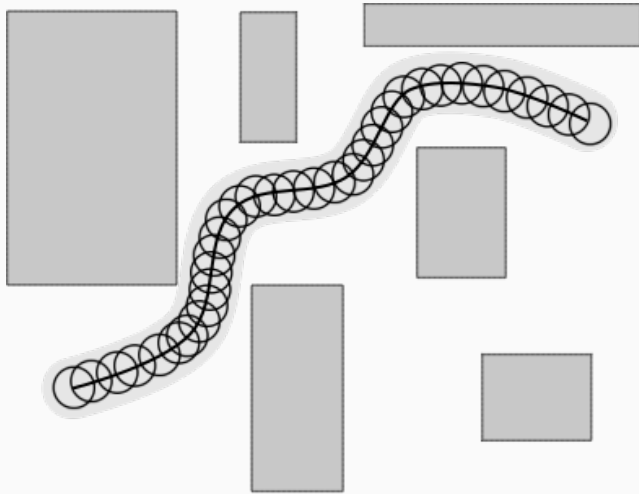
Assumption C: Sampling is dense.

Proof sketch

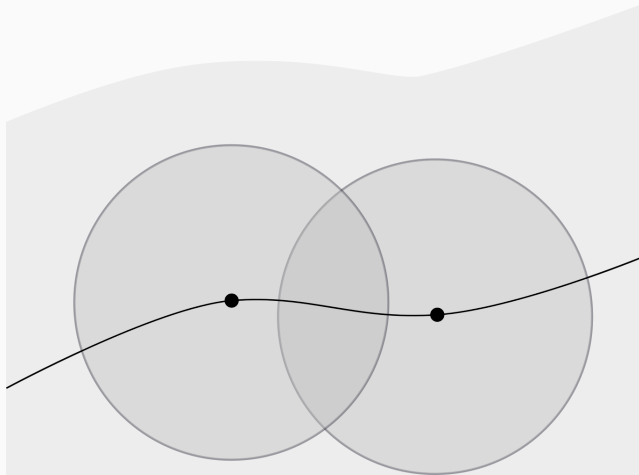


Assumption C: Sampling is dense.

Proof sketch

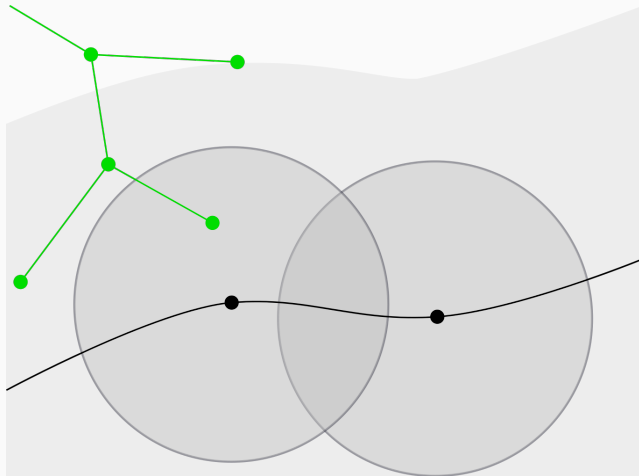


Step 1: Cover feasible path with δ -spaced discs.



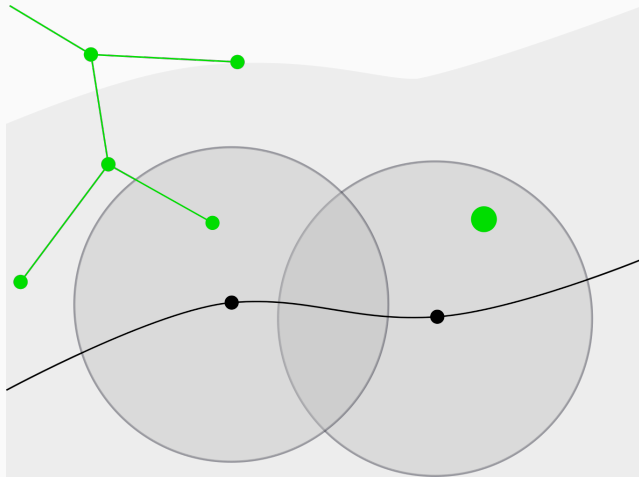
Step 2: Induction step (Base case is trivial)

Proof sketch



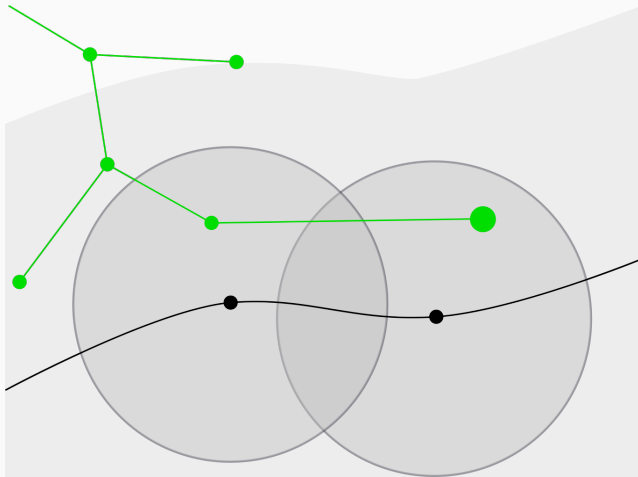
Step 2a: Assume we reached the n -th ball (Induction Assumption).
Need to prove that we reach $(n+1)$ -th ball.

Proof sketch



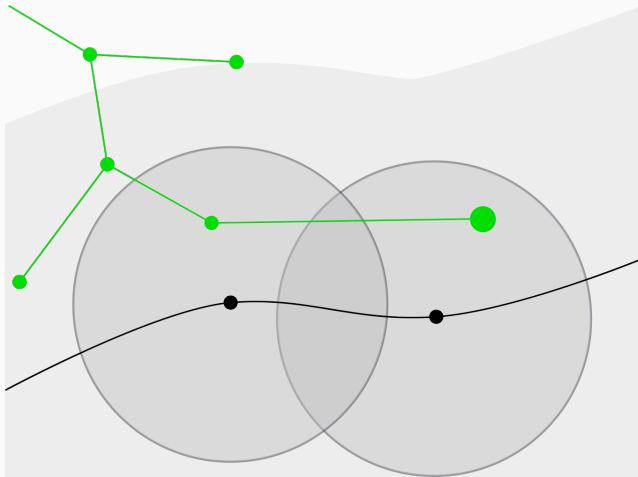
Step 2b: Sample in $(n+1)$ -th ball

Proof sketch



Step 2c: There exists a valid connection in free space

Proof sketch



This shows that you can construct a δ -similar path

Summary

- Assumption A: There is a feasible path
- Assumption B: It has ϵ clearance
- Assumption C: Sampling is dense

Proof sketch

- Put δ -spaced balls onto feasible path (depending on ϵ)
- Execute induction proof
 - Proof that the first ball is reached (trivial)
 - Proof that you reach ball B_{k+1} from B_k (main part)

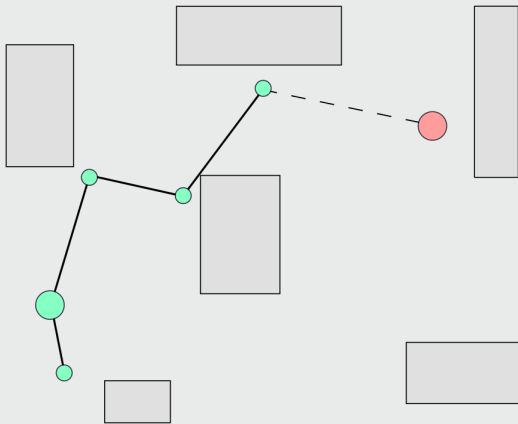
Rapidly-exploring random tree (RRT)

Improvements

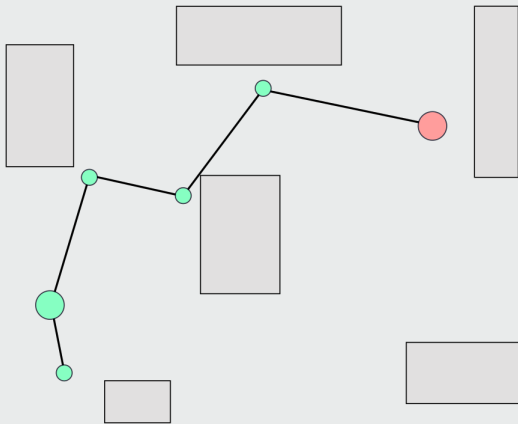
- Extend tree towards goal
- Sample goal region (with probability μ)
- Bidirectional tree

Improvement A - Extend towards goal

Extend towards goal

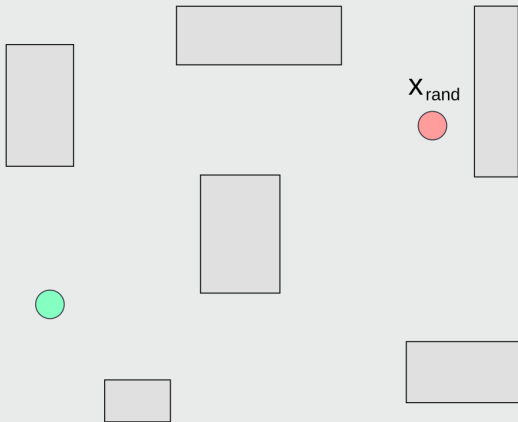


Extend towards goal



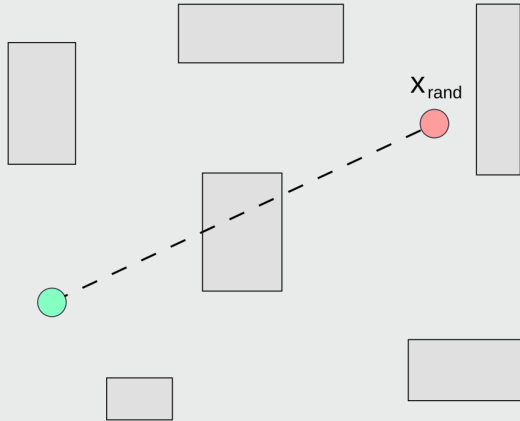
Improvement B - Sample goal region

Goal-bias



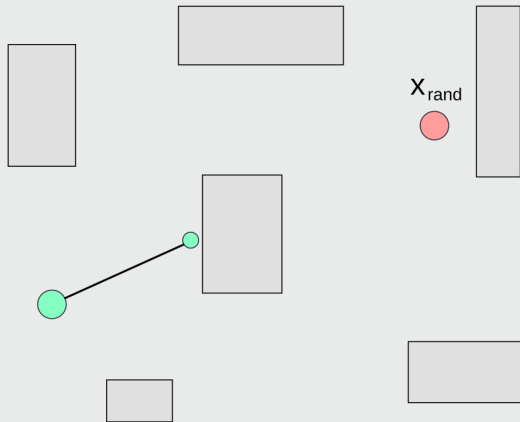
Improvement B - Sample goal region

Goal-bias



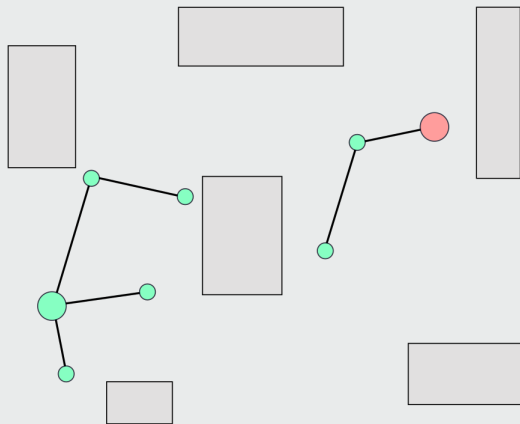
Improvement B - Sample goal region

Goal-bias



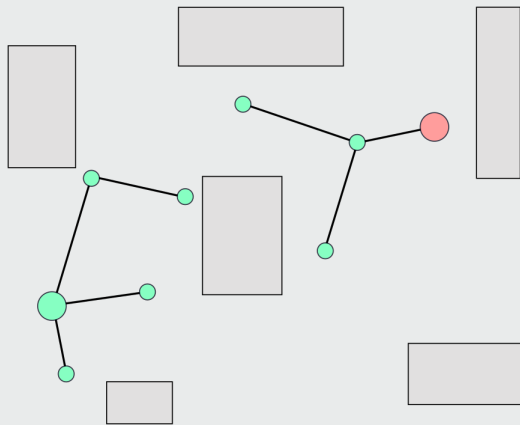
Improvement C - Bidirectional Tree

Bidirectional Rapidly-exploring random tree (Bi-RRT)



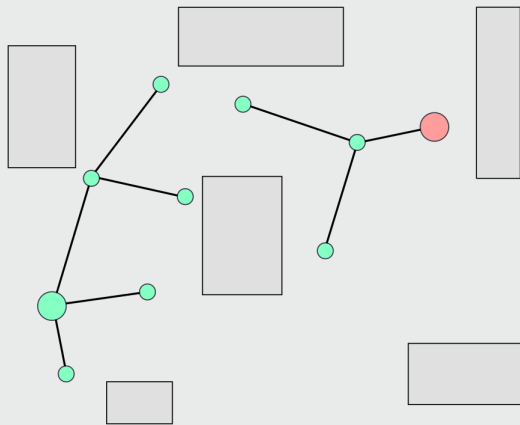
Improvement C - Bidirectional Tree

Bidirectional Rapidly-exploring random tree (Bi-RRT)



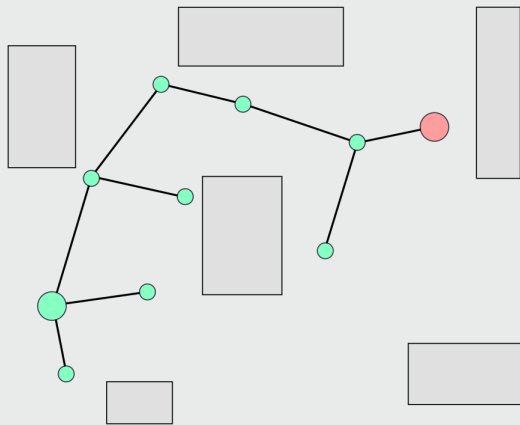
Improvement C - Bidirectional Tree

Bidirectional Rapidly-exploring random tree (Bi-RRT)



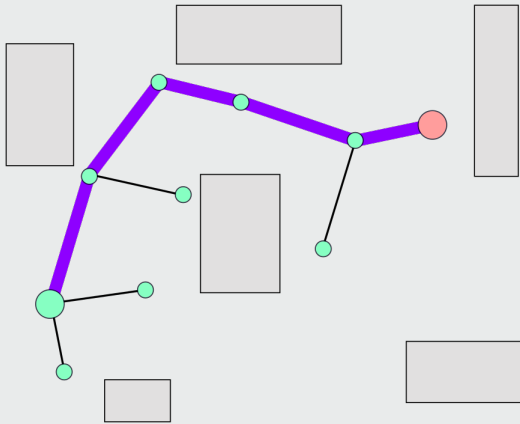
Improvement C - Bidirectional Tree

Bidirectional Rapidly-exploring random tree (Bi-RRT)



Improvement C - Bidirectional Tree

Bidirectional Rapidly-exploring random tree (Bi-RRT)



Further Improvements

- Path shortening after solution is found
- Multi-tree extension
- Targeted sampling

Introduction to Asymptotic Optimality Planning

Question

What is optimality?

Question

What is optimality?

Optimality (High-level)

The property of a planner to return a motion which surpasses all other motions in quality.

Question

What is optimality?

Optimality (Mid-level)

From all possible paths, return the one which minimizes an objective function.

Question

What is optimality?

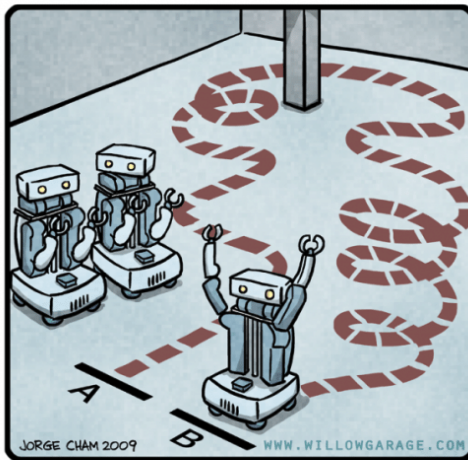
Optimality (Low-level)

Given a motion planning problem $\mathcal{Q}, q_I, q_G$, find a solution path p^* , which minimizes an objective cost function c , i.e. $c(p^*) \leq c(p)$ for all p which solve the problem.

Question

Why do we need optimality?

R.O.B.O.T. Comics



"HIS PATH-PLANNING MAY BE
SUB-OPTIMAL, BUT IT'S GOT FLAIR."

Efficiency







Usefulness of Optimality

- Aesthetics: Should look good from an observer perspective
- Efficiency: Should find time optimal paths
- Safety: Should keep distance to prevent collisions
- Coverage: Should reach every point of the workspace

Optimality principles also help us to search efficiently [1].

- A* heuristic: Prioritization search of best-cost paths VS. brute force search
- Pruning using necessary conditions

Introduction to Asymptotic Optimality Planning

Cost framework

Cost function types

Objective (or cost) function c . Graph $G = (V, E)$ and paths $P = (e_1, \dots, e_N)$.

- Cost for a configuration $c : V \rightarrow \mathbb{R}_{\geq 0}$
- Cost of an edge $c : E \rightarrow \mathbb{R}_{\geq 0}$
- Cost of a path $c : P \rightarrow \mathbb{R}_{\geq 0}$

Shortest length

- Configuration cost: Zero
- Edge cost: Length of segment, metric distance
- Path cost: Sum of edge costs

Maximum clearance

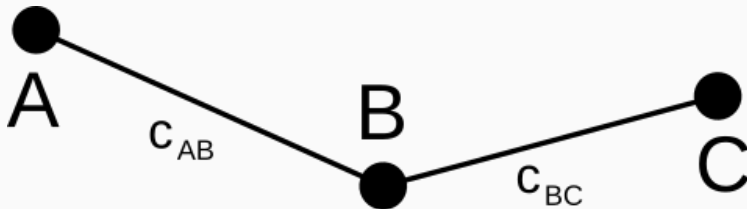
- Configuration cost: Distance from robot to environment
- Edge cost: Maximum over all configurations on edge
- Path cost: Maximum over all edges on path

Lowest energy

- Configuration cost: Zero
- Edge cost: Energy spent going from A to B
- Path cost: Sum of edge energies

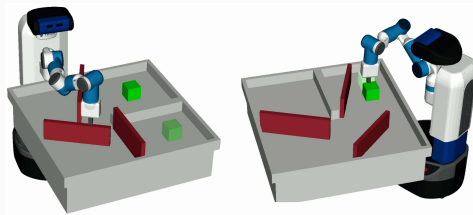
Additive costs

- Note: Most planners like RRT*, BIT* require additive cost!
- Additive cost: $\text{cost}(A,B,C) = \text{cost}(A,B) + \text{cost}(B,C)$



Non-additive cost example

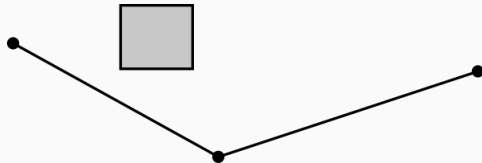
Number of objects manipulated by a robot manipulator



Bayraktar et al., "Solving Rearrangement Puzzles using Path Defragmentation in Factored State Spaces", Robotics and Automation Letters (RA-L), 2023

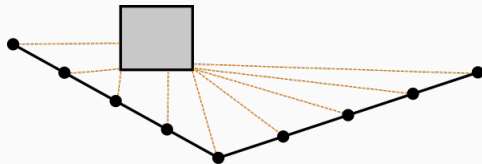
Non-additive cost example

Average clearance cost.



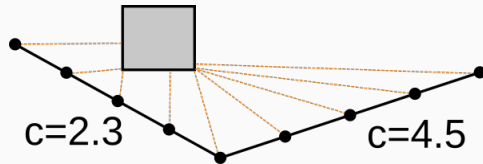
Non-additive cost example

Average clearance cost.



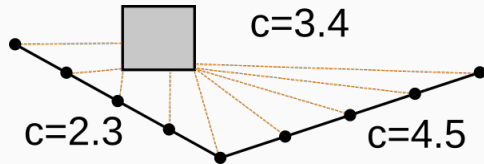
Non-additive cost example

Average clearance cost.



Non-additive cost example

Average clearance cost.



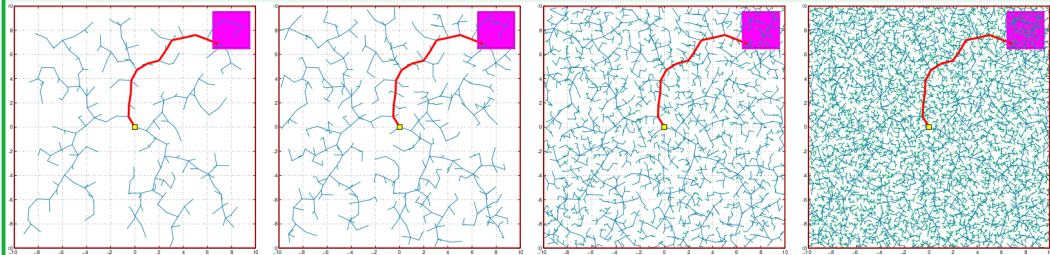
Cost framework recap

- What: Best possible motion
- Why: Aesthetics, Efficiency, Safety, Coverag, Optimality for efficient search
- How: Cost framework, additive costs

Optimal tree-based motion planning

RRT and Optimality (1)

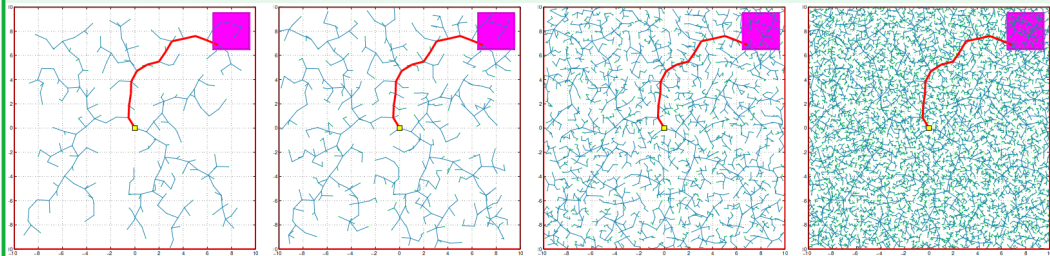
What if we keep running RRT?



Source: [2]

RRT and Optimality (1)

What if we keep running RRT?

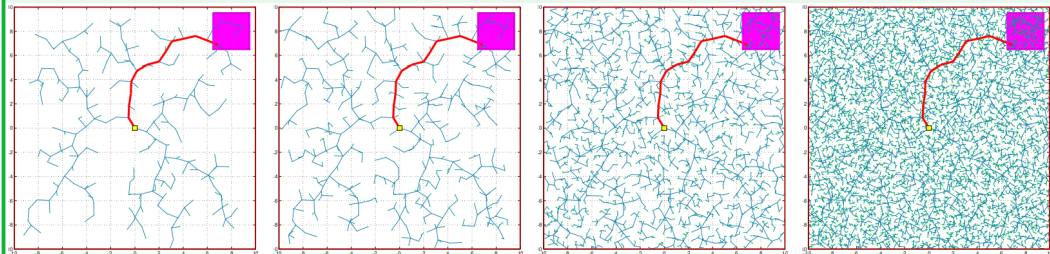


Source: [2]

What causes this?

RRT and Optimality (1)

What if we keep running RRT?



Source: [2]

What causes this?

Edges are only added, never changed (**rewired**).

[Sampling-based algorithms for optimal motion planning](#)

[S Karaman](#), [E Frazzoli](#) - The international journal of robotics ..., 2011 - journals.sagepub.com

During the last decade, sampling-based path planning algorithms, such as probabilistic roadmaps (PRM) and rapidly exploring random trees (RRT), have been shown to work well ...

☆ Save  Cite **Cited by 3551** Related articles

RRT and Optimality (2)

[Sampling-based algorithms for optimal motion planning](#)

[S Karaman](#), [E Frazzoli](#) - The international journal of robotics ..., 2011 - journals.sagepub.com

During the last decade, sampling-based path planning algorithms, such as probabilistic roadmaps (PRM) and rapidly exploring random trees (RRT), have been shown to work well ...

☆ Save  Cite **Cited by 3551** Related articles

RRT is Suboptimal [2, Theorem 33]

The cost of the best solution returned by RRT converges to a suboptimal value, with probability one:

$$\mathbb{P} \left(\left\{ \lim_{n \rightarrow \infty} Y_n^{RRT} > c^* \right\} \right) = 1.$$

RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [3]

RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [3]
- Parent of $\mathbf{q}_{\text{new}}$: May use other parent than $\mathbf{q}_{\text{near}}$ with lowest cost (within neighborhood of $\mathbf{q}_{\text{new}}$)

RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [3]
- Parent of q_{new} : May use other parent than q_{near} with lowest cost (within neighborhood of q_{new})
- Rewire edges: Use q_{new} as a new parent, for neighboring configurations, if it reduces costs

RRT*: RRT with Rewiring

Algorithm 2 RRT* ($x_{\text{init}} := s, x_{\text{goal}} := t, n, r, \eta$)

```
1:  $V = \{x_{\text{init}}\}$ 
2: for  $j = 1$  to  $n$  do
3:    $x_{\text{rand}} \leftarrow \text{SAMPLE-FREE}()$ 
4:    $x_{\text{near}} \leftarrow \text{NEAREST}(x_{\text{rand}}, V)$ 
5:    $x_{\text{new}} \leftarrow \text{STEER}(x_{\text{near}}, x_{\text{rand}}, \eta)$ 
6:   if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
7:      $X_{\text{near}} = \text{NEAR}(x_{\text{new}}, V, \min\{r(|V|), \eta\})$ 
8:      $V = V \cup \{x_{\text{new}}\}$ 
9:      $x_{\text{min}} = x_{\text{near}}$ 
10:     $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
11:    for  $x_{\text{near}} \in X_{\text{near}}$  do
12:      if  $\text{COLLISION-FREE}(x_{\text{near}}, x_{\text{new}})$  then
13:        if  $\text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\| < c_{\text{min}}$  then
14:           $x_{\text{min}} = x_{\text{near}}$ 
15:           $c_{\text{min}} = \text{COST}(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|$ 
16:     $E = E \cup \{(x_{\text{min}}, x_{\text{new}})\}$ 
17:    for  $x_{\text{near}} \in X_{\text{near}}$  do
18:      if  $\text{COLLISION-FREE}(x_{\text{new}}, x_{\text{near}})$  then
19:        if  $\text{COST}(x_{\text{new}}) + \|x_{\text{near}} - x_{\text{new}}\| < \text{COST}(x_{\text{near}})$  then
20:           $x_{\text{parent}} = \text{PARENT}(x_{\text{near}})$ 
21:           $E = E \cup \{(x_{\text{new}}, x_{\text{near}})\} \setminus \{(x_{\text{parent}}, x_{\text{near}})\}$ 
22: return  $G = (V, E)$ 
```

- Pseudo code from [3]
- Parent of $\mathbf{q}_{\text{new}}$: May use other parent than $\mathbf{q}_{\text{near}}$ with lowest cost (within neighborhood of $\mathbf{q}_{\text{new}}$)
- Rewire edges: Use $\mathbf{q}_{\text{new}}$ as a new parent, for neighboring configurations, if it reduces costs
- Neighborhood radius depends on tree size:

$$r(|\mathcal{V}|) = \gamma \left(\frac{\log |\mathcal{V}|}{|\mathcal{V}|} \right)^{\frac{1}{d+1}}$$

Rewiring

If we add a new configuration x , we execute two rewiring operations:

- Rewire x to best parent
- Rewire all children nodes

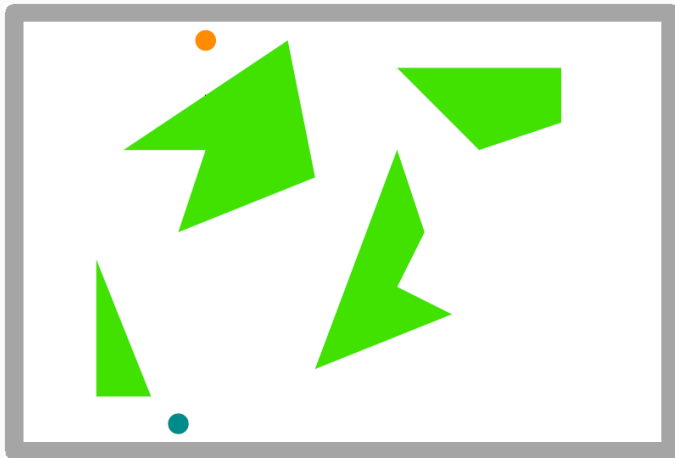
Pseudocode tree rewiring

```
1  def Rewire(x):
2      N = Neighbours(x)
3      for x_n in N:
4          Rewire(x_n, x)
5      for x_n in N:
6          Rewire(x, x_n)
7
8  def Rewire(x, y):
9      p = Steer(x, y)
10     if ConstraintFree(p):
11         if cost(x)+cost(p) < cost(y):
12             y.parent = x
```

Pseudocode RRT

```
1 def RRT(xstart, xgoal, mu):
2     V.AddNode(xstart)
3     while not finished:
4         xrand = SampleRandom()
5         xnear = NearestNeighbor(xrand)
6         xnew = Steer(xnear, xrand, mu)
7         if xnear == xnew:
8             continue
9         V.AddNode(xnew)
10        V.AddEdge(xnear, xnew)
11        Rewire(xnew) ##Rewiring operation to make it AO
12        if Distance(xnew, xgoal) < Epsilon:
13            return Path(xnew)
```

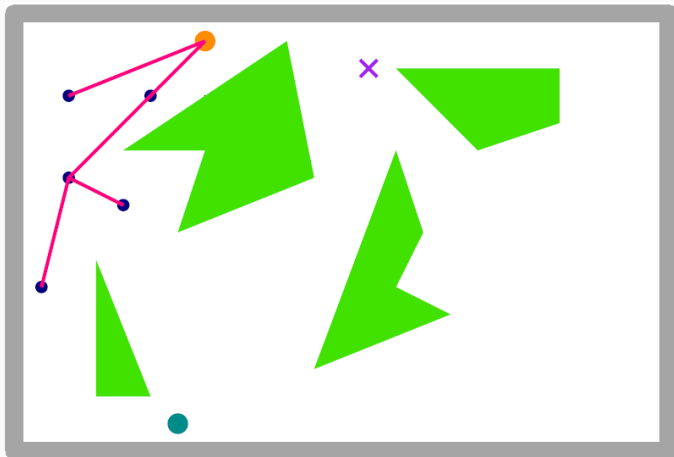
RRT* Example (1)



Source: [4]

Motion planning problem (orange = $\mathbf{q}_{start}$)

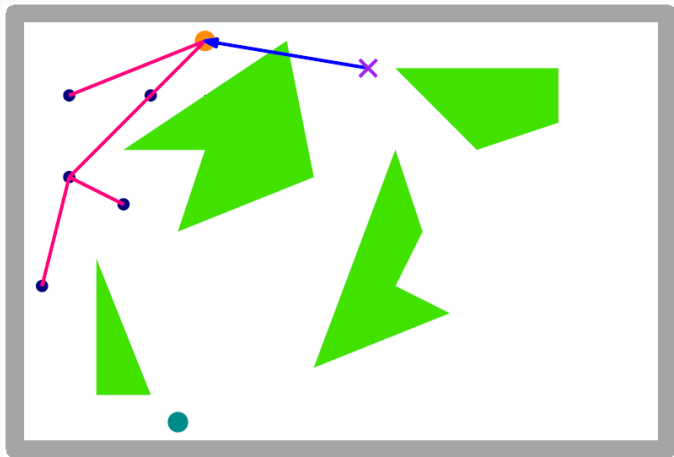
RRT* Example (2)



Source: [4]

Intermediate tree and new sample $\mathbf{q}_{rand}$ (purple $\times$)

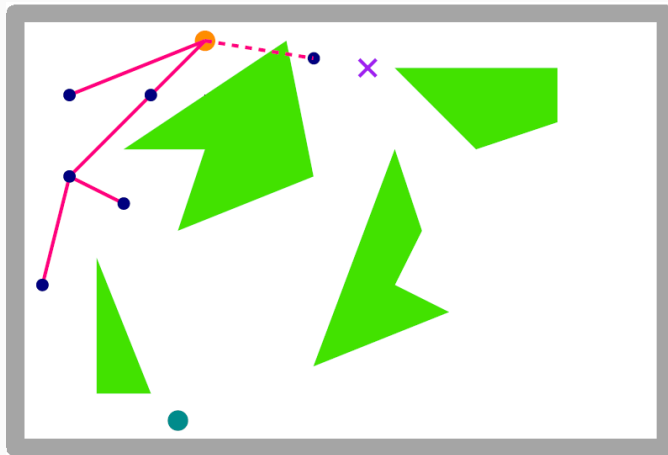
RRT* Example (3)



Source: [4]

Nearest $\mathbf{q}_{near}$ in existing tree is found (here: $\mathbf{q}_{start}$)

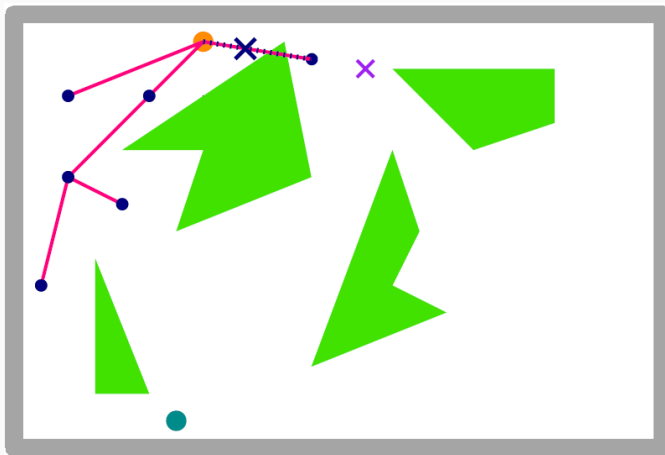
RRT* Example (4)



Source: [4]

Steer computes q_{new} on the line from q_{start} to q_{rand}

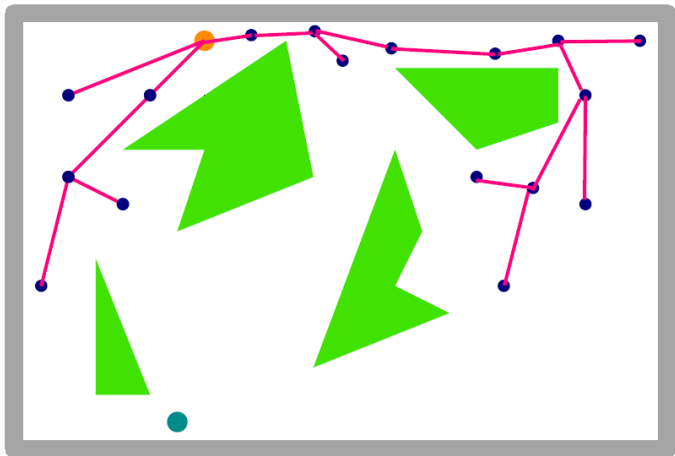
RRT* Example (5)



Source: [4]

New edge is rejected (not collision-free)

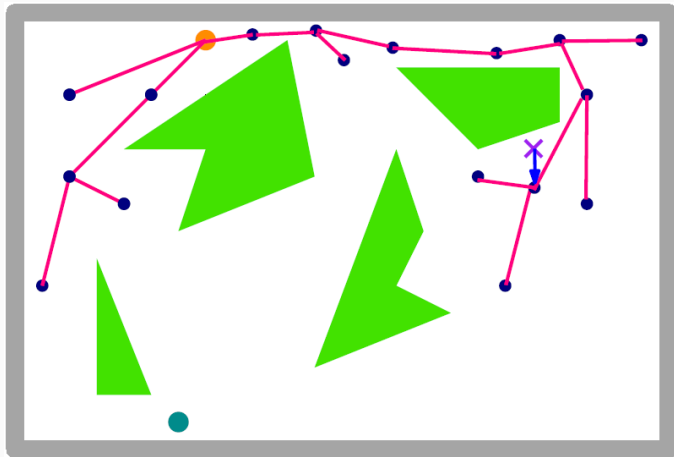
RRT* Example (6)



Source: [4]

So far behavior is exactly the same as RRT; Fast-forward we have a larger tree

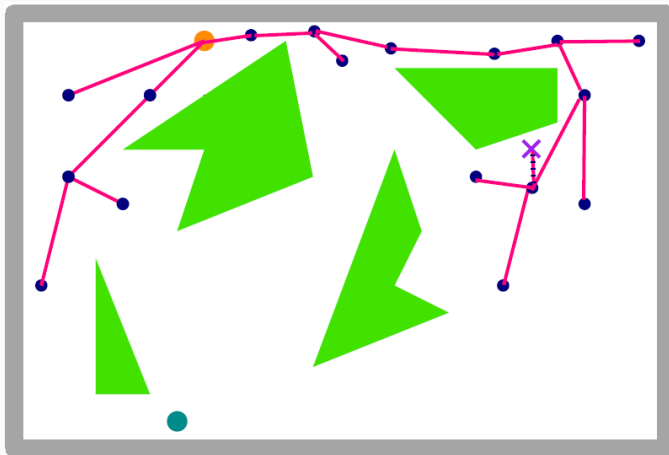
RRT* Example (7)



Source: [4]

New sample $\mathbf{q}_{rand}$ and closest node in the tree $\mathbf{q}_{near}$

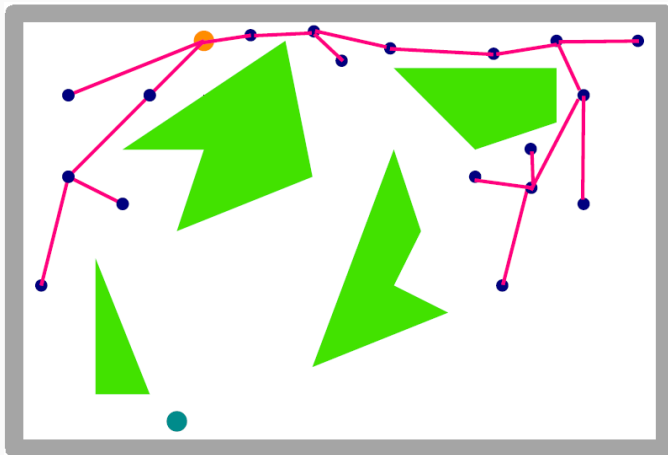
RRT* Example (8)



Source: [4]

Resulting edge ($\mathbf{q}_{new}$, $\mathbf{q}_{near}$) is collision-free

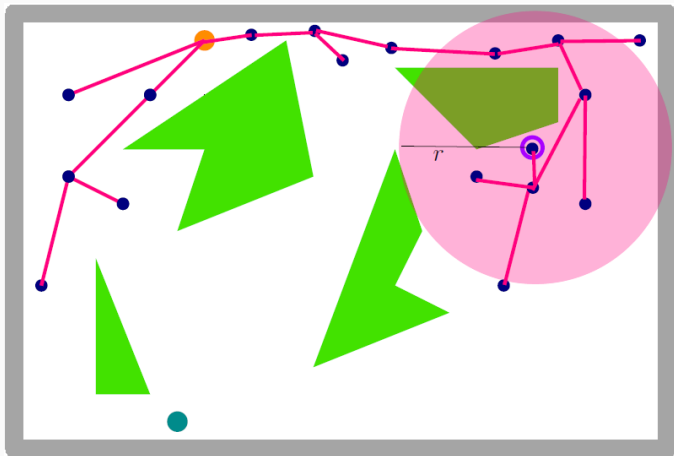
RRT* Example (9)



Source: [4]

This edge would be added in RRT

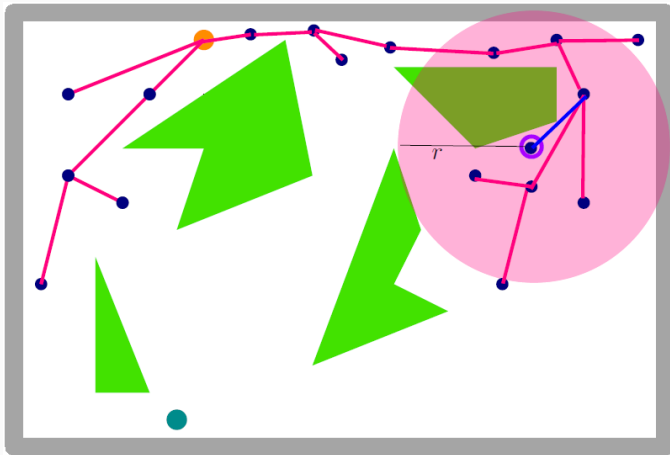
RRT* Example (10)



Source: [4]

RRT*: Consider all configuration of the tree in the neighborhood of $\mathbf{q}_{new}$

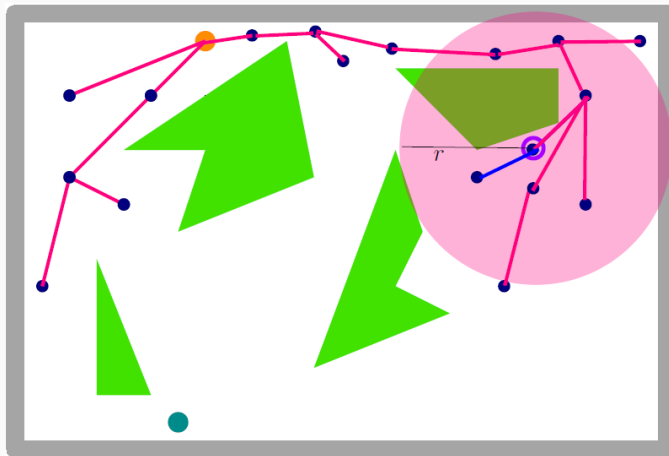
RRT* Example (11)



Source: [4]

RRT*: Use a lower-cost parent for $\mathbf{q}_{new}$ (other than $\mathbf{q}_{near}$)

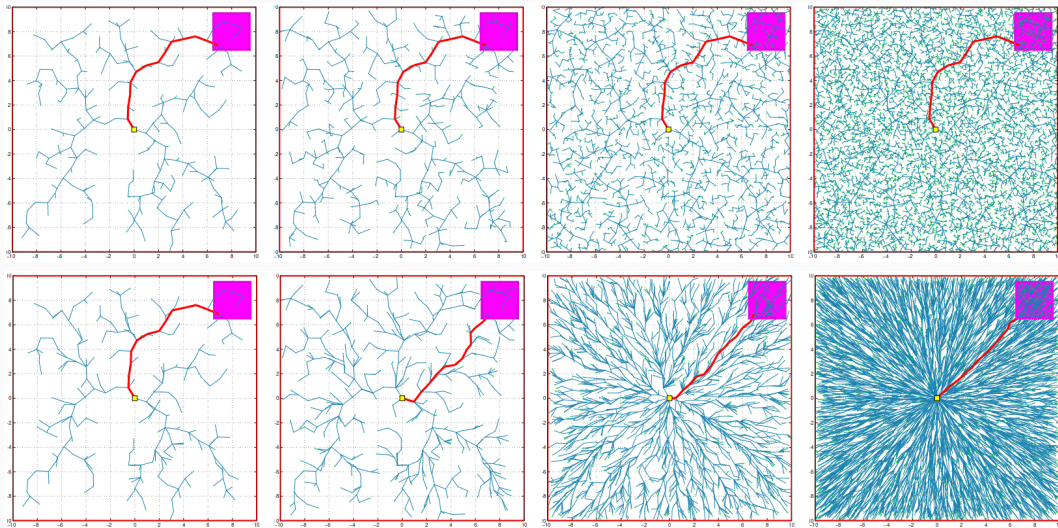
RRT* Example (12)



Source: [4]

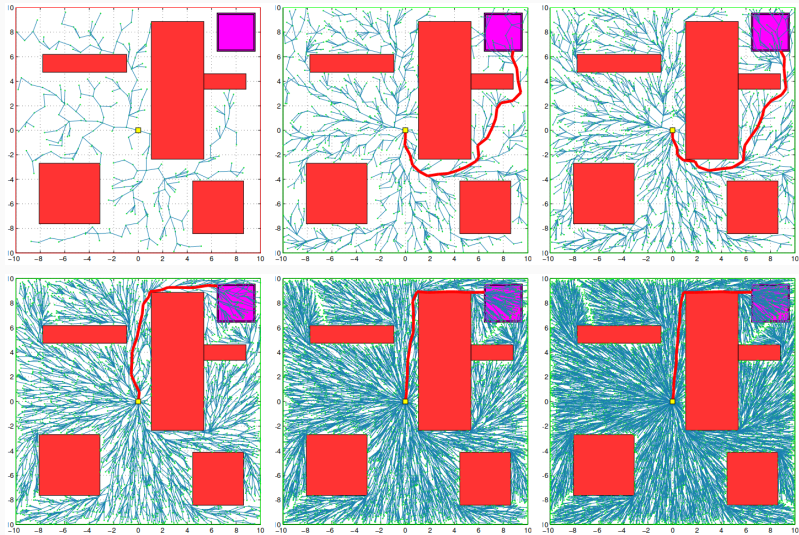
RRT*: Rewire the neighbors to use $\mathbf{q}_{new}$ as a parent to reduce the cost

RRT vs. RRT* (1)



Source: [2]

RRT vs. RRT* (2)



Source: [2]

RRT* is asymptotically optimal

The probability that the solution cost of RRT* is not more than $(1 + \epsilon)c^*$ is 1, as the number of iterations go to infinity:

$$\lim_{n \rightarrow \infty} \mathbb{P}(\{c_n - c^* > \epsilon\}) = 0, \forall \epsilon > 0.$$

RRT* is asymptotically optimal

The probability that the solution cost of RRT* is not more than $(1 + \epsilon)c^*$ is 1, as the number of iterations go to infinity:

$$\lim_{n \rightarrow \infty} \mathbb{P}(\{c_n - c^* > \epsilon\}) = 0, \forall \epsilon > 0.$$

However, the convergence rate is unknown!

Kinodynamic RRT

Kinodynamic Planning



(Optimal) Kinodynamic Planning

Given

- State space $\mathcal{Q}$ and free state space $\mathcal{Q}_{free}$
- Control space $\mathcal{U}$
- Dynamics $\dot{\mathbf{q}}(t) = \mathbf{f}(\mathbf{q}(t), \mathbf{u}(t))$
- Initial state $\mathbf{q}_{start} \in \mathcal{Q}_{free}$
- Goal region $\mathcal{Q}_{goal} \subset \mathcal{Q}_{free}$
- Cost $c = J(T, \mathbf{u}(t), \mathbf{q}(t))$

Desired

- Trajectory $\pi : [0, T] \rightarrow \mathcal{Q}_{free} \times \mathcal{U}$

- Feasible kinodynamic planning: Compute a trajectory π
- Optimal kinodynamic planning: From all feasible trajectories, select the one which minimizes the cost (we call it π^*)

Kinodynamic RRT

Steering vs Forward Propagation

Two variants

Two types of kinodynamic planning depending on information available

- Steering
- Forward propagation

Steering vs Propagation

Planners require access to dynamical function $\mathbf{f}$. This can be accomplished in two ways

- Steering: Given two states $\mathbf{q}$, $\mathbf{q}'$, compute controls to move robot from $\mathbf{q}$ to $\mathbf{q}'$
 - Involves solving a boundary-value problem (BVP)
 - Computationally expensive
 - Tricky if dynamical constraints are involved
- Forward Propagation: Given a state $\mathbf{q}$, a control $\mathbf{u}$, and a time Δt , compute the next state $\mathbf{q}'$ by applying control $\mathbf{u}$ for time Δt
 - Simple to compute
 - Does not require knowledge of system
 - Unclear how to use it for (optimal) planning

Steering function

Planning with steering functions as generalized interpolation

- Reduction to geometric case
- Any geometric planner can be applied
- PRM, RRT*, or BIT*

Planning with forward propagation

- Difficult: Unclear how to exploit forward propagation
- How to make this optimal?
- Optimal kinodynamic motion planning: Naive random trees and SST*

Kinodynamic RRT

Meta algorithm

Meta Algorithm (for kinodynamic planning with forward propagation)

Meta($\mathbf{q}_{start}$, $\mathcal{Q}_{goal}$, $\mathcal{Q}$, $\mathcal{U}$, $\mathbf{f}$, t_{prop})

- $T = \text{InitializeTree}(\mathbf{q}_{start})$
- While Not Terminated
 - $\mathbf{q}_{select} = \text{SelectNode}(T, \mathcal{Q})$
 - $\mathbf{q}_{new} = \text{Propagate}(\mathbf{q}_{select}, \mathcal{U}, \mathbf{f}, t_{prop})$
 - $\text{MaybeAddConnection}(\mathbf{q}_{select}, \mathbf{q}_{new})$

Note: In practice, we would terminate either after N iterations, or when a path is found, or when a certain cost is reached, etc. For the theoretical analysis, however, we assume that the algorithm will not terminate (asymptotic optimality can only be reached in the limit).

Select Node

- Uniform Selection: Pick a node from the graph at random
- Exploration First: Pick a node which increases explorative nature of algorithm (cover state space as quickly as possible)
- Best First: Pick a node on a high-quality path

Propagate Node

MaybeAddConnection

Select Node

Propagate Node

- Fixed Duration: Pick random controls, then apply them for a fixed time t_{prop}
- Monte-Carlo: Pick random control and random time, then propagate system forward
- Guided Monte-Carlo ("shooting" method): Select random target state. Sample k controls and k times. Propagate them forward and select the node nearest to target as return value.

MaybeAddConnection

Select Node

Propagate Node

MaybeAddConnection

- Collision-free: Add connection if no collision occurred
- Prune dominated: Add connection if collision free and locally having the best cost.

Instantiations of Meta algorithm

Algorithms differ in how they implement the three modules "Select Node", "Propagate Node", and "MaybeAddConnection".

- Kinodynamic RRT (kRRT)
- Naive Random Trees (NRT)
- Stable sparse trees (SST*)

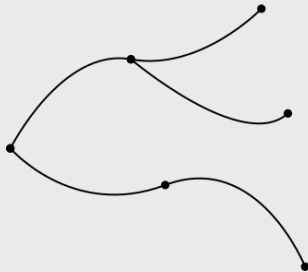
Kinodynamic RRT

Kinodynamic RRT

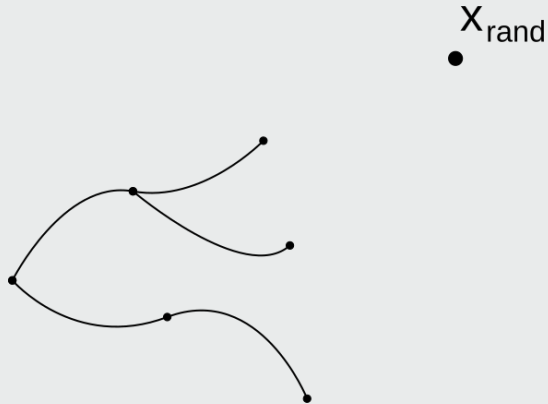
Kinodynamic RRT

- Select Node: Exploration First Selection
- Propagate Node: Guided Monte Carlo Propagation
- Maybe add connection: Collision-Free Checking

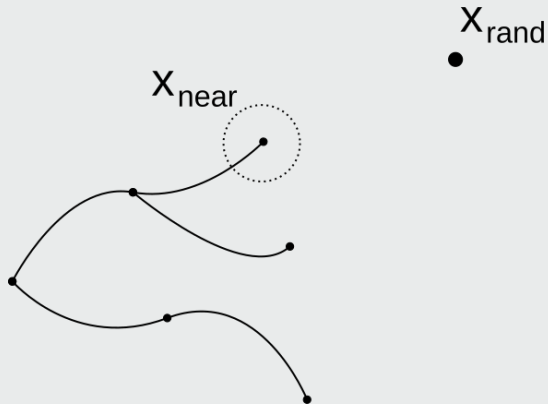
Select Node



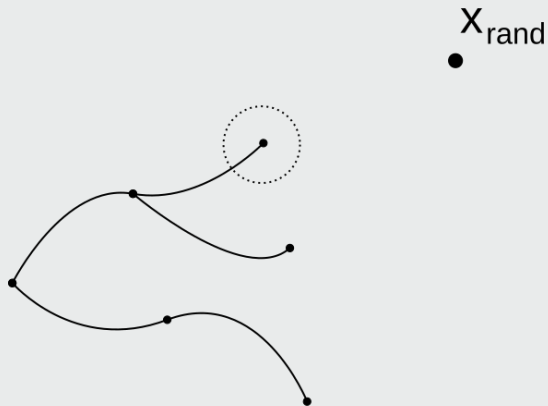
Select Node



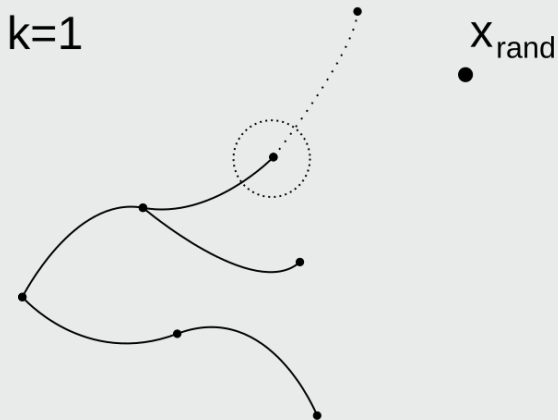
Select Node



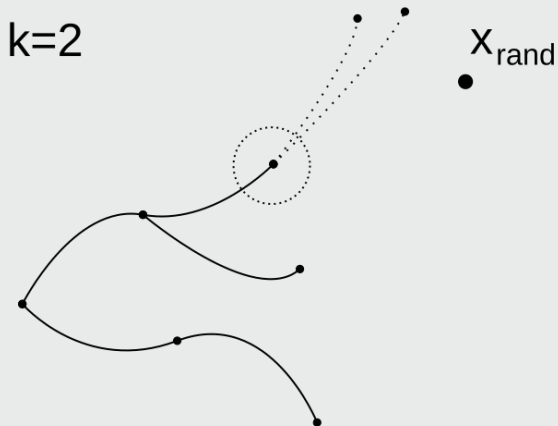
Propagate Node



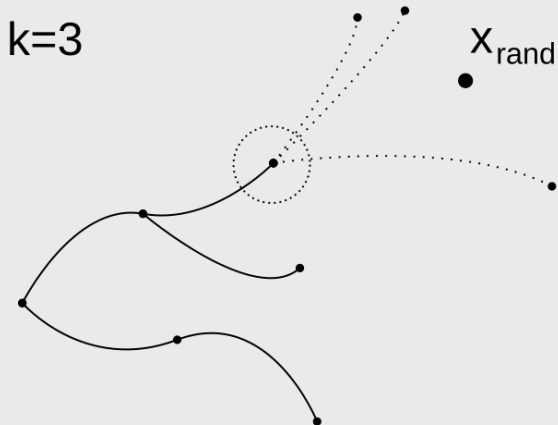
Propagate Node



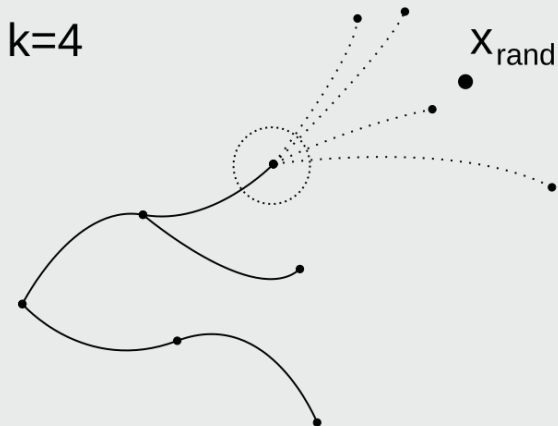
Propagate Node



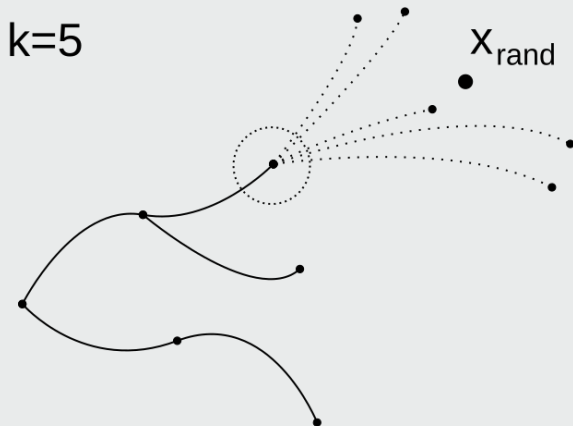
Propagate Node



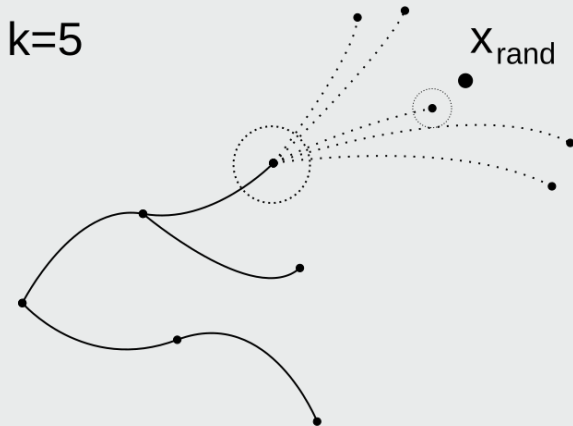
Propagate Node



Propagate Node



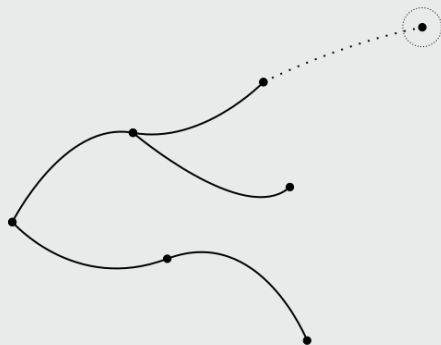
Propagate Node



Propagate Node

$k=5$

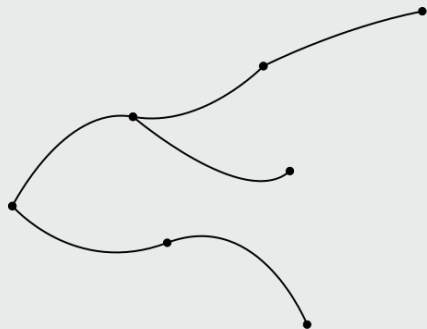
x_{rand}



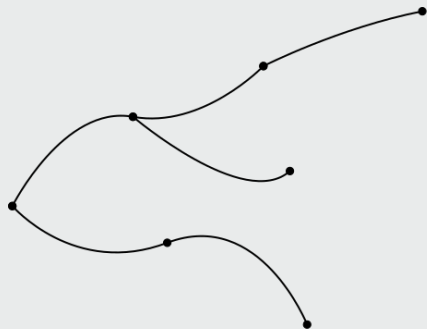
Maybe Add Connection

$k=5$

x_{rand}



Maybe Add Connection



Properties

Kinodynamic RRT is probabilistically complete*

*For specific classes of dynamical systems.

LaValle and Kuffner, "Randomized Kinodynamic Planning", 2001

Kleinbort et al., "Probabilistic completeness of RRT for geometric and kinodynamic planning with forward propagation", 2022

Conclusion

- New geometric planners: RRT, RRT*
- New kinodynamic planner: kinodynamic RRT

Next time

- Informed RRT
- AO-RRT: Asymptotic optimality using cost extension
- SST*: Asymptotic optimality using forward propagation

- [1] Judea Pearl. **Heuristics: Intelligent Search Strategies for Computer Problem Solving**. 1984.
- [2] Sertac Karaman and Emilio Frazzoli. **“Sampling-Based Algorithms for Optimal Motion Planning”**. In: *International Journal of Robotics Research (IJRR)* 30.7 (2011), pp. 846–894. DOI: 10.1177/0278364911406761.
- [3] Kiril Solovey, Lucas Janson, Edward Schmerling, Emilio Frazzoli, and Marco Pavone. **“Revisiting the Asymptotic Optimality of RRT*”**. In: *IEEE International Conference on Robotics and Automation (ICRA)*. May 2020, pp. 2189–2195. DOI: 10.1109/ICRA40945.2020.9196553.

- [4] Dan Halperin. **Algorithmic Robotics and Motion Planning**. 2020. URL: <http://acg.cs.tau.ac.il/courses/algorithmic-robotics/fall-2019-2020/algorithmic-robotics-and-motion-planning>.
- [5] Steven M. LaValle. **Planning algorithms**. Cambridge University Press, 2006. ISBN: 978-0-521-86205-9. URL: <http://planning.cs.uiuc.edu>.